LRBAC Writeup

Matthew Hays and Nicholas Bava

August 2025

1 Design Progression

Initial exploration of the project began by reviewing the documentation that accompanied the assignment [1, 2, 3]. This work formed the basis of our initial literature review and deep dive into RBAC specifications. The exploration then led us to the NIST documentation that clarifies the purpose of each primary RBAC artifact [4]. After developing a better understanding of the functionality provided at the four RBAC levels, $RBAC_0$, $RBAC_1$, $RBAC_2$, and $RBAC_3$, we began implementing the model described in [2].

1.1 Initial Paper Design

Ray et al. [2] describes an RBAC model with explicit, admin-defined separation-of-duty mechanisms: static separation of duties (SSD) and dynamic separation of duties (DSD).

- SSD is a *assignment-time* check that "a user may not be assigned two conflicting roles defined within the role's SSD set."
- DSD is a run-time (*session-time*) check: "a user may be assigned two roles defined in the role's DSD; however, conflicting roles cannot be activated within the same session."

Implementation of the Ray et al. [2] model was performed. However, because of our interpretation of the assignment's intent, we moved to the extended model in [1]. For completeness, our progress with the original design is preserved in the report *RBAC_Writeup_INITIAL_DRAFT.pdf*.

Relevant Artifacts

- User
- Session
- Permission
- Role

1.2 History- and Resource-Based DSOD Design

Kuhlmann et al. [1] extend the earlier RBAC model to offer finer-grained control over resources, roles, sessions, and snapshots. Instead of allowing static SSD/DSD lists to propagate automatically, constraint propagation is governed by Boolean flags on the `MutuallyExclusive` class:

- `wrtUserAssignment` = `true` ⇒ enable SSD checks.
- `wrtActiveRoles` = `true` ⇒ enable DSD checks.
- `wrtJuniors` / `wrtSeniors` control whether a constraint also applies to junior or senior roles, respectively.
- `identicalSeniorAllowed` = `true` ⇒ lets two roles conflicting roles that share a common senior role bypass SSD conflicts if the user is assigned to that senior.
- `wrtPermissionAssignment` = `true` ⇒ for two roles linked by the same `MutuallyExclusive` object whose flag is set, the same permission may not be assigned to both roles.

This flag-based approach supports additional variants such as history-based or resource-based DSD without redefining the entire role hierarchy, giving administrators precise, context-aware control over separation-of-duty policies.

Relevant Artifacts

- User
- Session
- Permission
- Role
- MutuallyExclusive
- Snapshot
- Access
- Activation

Class Diagram

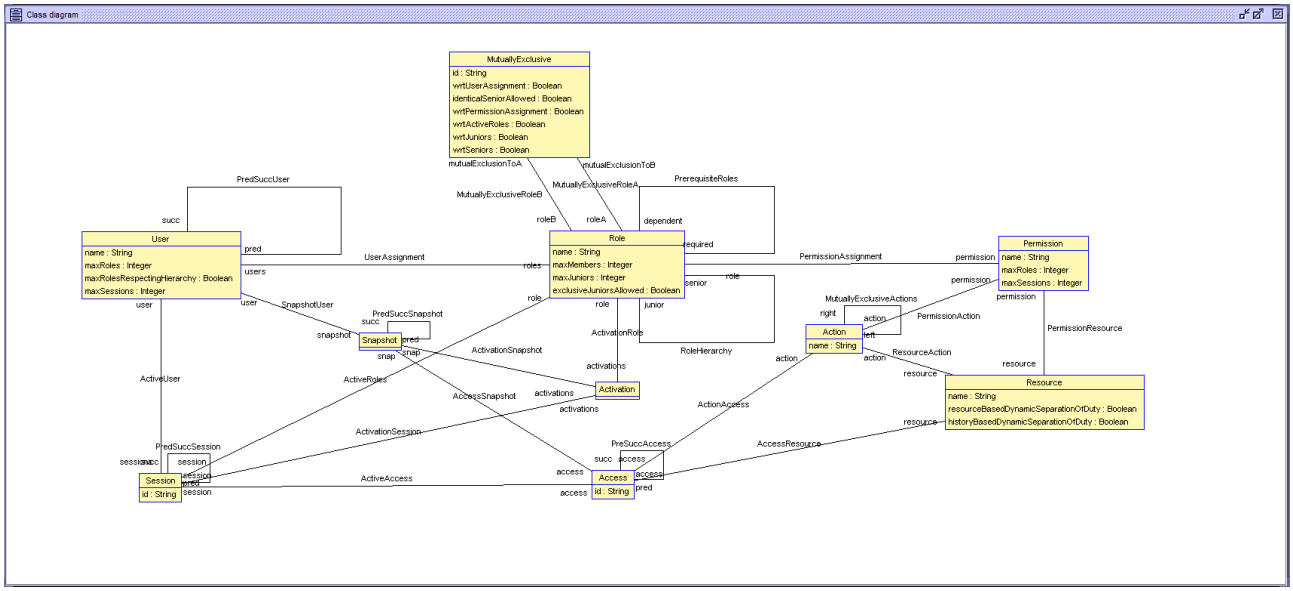

Figure 1: Class Diagram

2 Role Hierarchies, SSD, and DSD Impacts

2.1 Role Hierarchies

In the base model defined in Ray et al. [2], role hierarchies are defined by `junior` and `senior` role relationships between two (or more) roles. This allows an *inheritance chain* of roles where `senior` roles inherit from `junior` roles. While on the surface, this inheritance direction seems backwards from the object-oriented approach where children inherit from parents, senior roles contain more privileges defined on top of junior roles. As a result, `senior` roles may be thought of as *child* classes in contrast to typical object-oriented semantics.

As `senior` roles inherit from `junior` roles, SSD and DSD constraints defined on `junior` roles automatically propagate up to `senior` roles. In other words, if an SSD or DSD constraint exists on a `junior` role, then it automatically applies to all of its `senior` roles through inheritance.

However, the model defined in Kuhlmann et al. [1] approaches hierarchical relationships in a more granular way. In this model, we define hierarchies using the `RoleHierarchy` association, as before, but propagation of mutual exclusion constraints is not automatic. Instead, it is governed by flags in the `MutuallyExclusive` class. Specifically, the `wrtJuniors` and `wrtSeniors` flags determine whether the exclusion constraints should propagate down or up the hierarchy. This allows constraint inheritance to be controlled on a per-exclusion basis. Note that this propagation applies only to mutual exclusion logic; permission inheritance from `junior` to `senior` roles still occurs universally.

Relevant Invariants and Operations:

- `Role::juniors()`, `Role::seniors()` define hierarchy traversal using transitive closure.
- `Role::userAssignmentExclusive()`, `activeRolesExclusives()` apply constraint propagation via flags.
- `AcyclicRoleHierarchy` enforces acyclic role hierarchies.

2.2 Static Separation of Duties

Starting with Ray et al. [2], the model defines a `SSD` relationship between roles. This collection is populated by the administrator or derived from policy to predefine conflicting roles at role-definition time. The constraint, in plain terms, states: *"No user may be assigned more than one role from within an SSD set."* This is an *assignment-time* constraint, where the restriction is enforced when a role is added to a `User`'s list of `roles`. This statically enforces policy by preventing violations before they occur.

While extremely similar in construction, the model proposed in Kuhlmann et al. [1] propagates SSD constraints on a per mutual exclusion relationship basis. Each mutual exclusion enables SSD behavior by setting the `wrtUserAssignment` flag to `true`. This allows administrators to control which mutual exclusions enforce SSD logic. Additionally, propagation of SSD conflicts across the role hierarchy is governed by the `wrtJuniors` and `wrtSeniors` flags.

Conflicts may optionally be bypassed if two conflicting roles share a common senior role *and the user is explicitly assigned to that senior role*, provided the `identicalSeniorAllowed` flag is also set to `true`. This allows policy exceptions when a user is entrusted with a high-level role that encompasses the responsibilities of both conflicting roles. This mechanism allows organizations to selectively override exclusion policies through deliberate assignment at higher levels of the role hierarchy.

Object Diagram for Test 1 - No Exclusive Roles Assigned

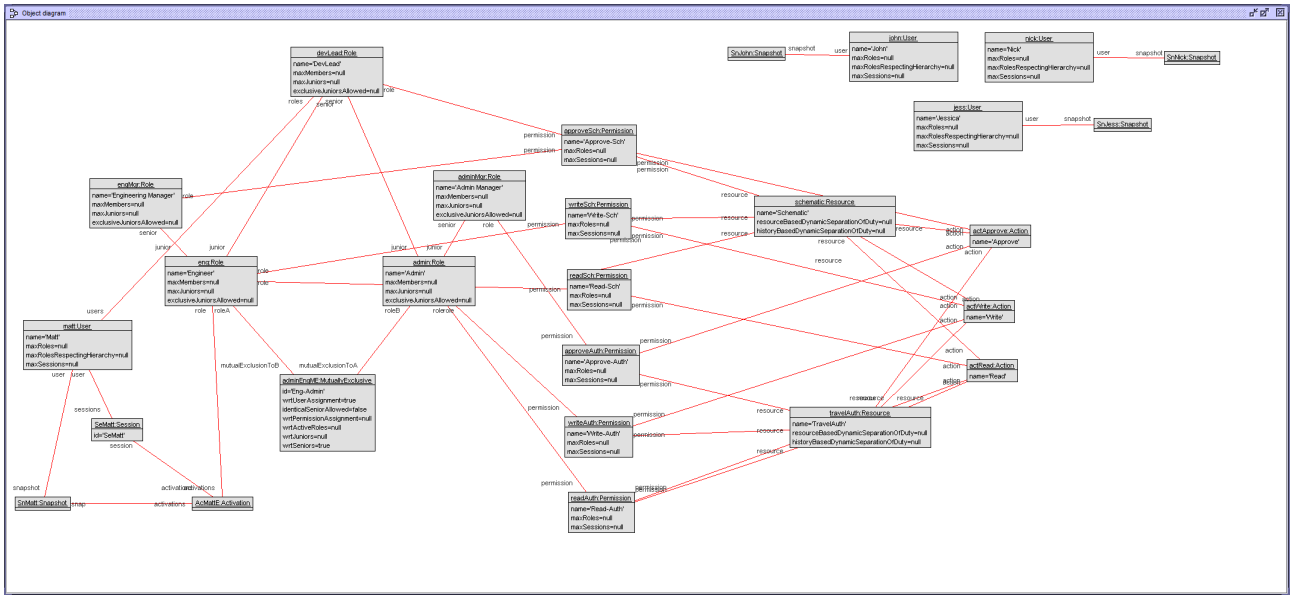

Figure 2: Object Diagram

Test Output for Test 1 - No Exclusive Roles Assigned

```
1.x> ----------------------------------------------------------
1.x> -- 8.  RUN THE CONSISTENCY CHECK  --------------------------
1.x> ----------------------------------------------------------
1.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (3) `Permission::PermissionMaxRoles': OK.
checking invariant (4) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (5) `Resource::HistoryBasedDSOD': OK.
checking invariant (6) `Resource::ResourceBasedDSOD': OK.
checking invariant (7) `Role::AcyclicRoleHierarchy': OK.
checking invariant (8) `Role::MaxJuniorsBound': OK.
checking invariant (9) `Role::MaxMembersBound': OK.
checking invariant (10) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (11) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (12) `Session::NoSessionCycle': OK.
checking invariant (13) `User::MaxSessionsPerUser': OK.
checking invariant (14) `User::NoExclusiveRolesAssigned': OK.
checking invariant (15) `User::NoUserCycle': OK.
checked 15 invariants in 0.024s, 0 failures.
1.x> --  Γ£ö 0 structural problems
1.x> --  Γ£ö all 15 invariants satisfied
1.x> ? SeMatt.checkAccess(actWrite, travelAuth, SnMatt) -- Expect False --> admin is not active in Matt's session
-> false : Boolean
1.x> ? SeMatt.checkAccess(actWrite, schematic, SnMatt) -- Expect True
-> true : Boolean
1.x> ? SeMatt.checkAccess(actApprove, schematic, SnMatt) -- Expect False --> engMgr required
-> false : Boolean
1.x>
1.x> !set adminEngME.identicalSeniorAllowed := false
1.x>
1.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (3) `Permission::PermissionMaxRoles': OK.
checking invariant (4) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (5) `Resource::HistoryBasedDSOD': OK.
checking invariant (6) `Resource::ResourceBasedDSOD': OK.
checking invariant (7) `Role::AcyclicRoleHierarchy': OK.
checking invariant (8) `Role::MaxJuniorsBound': OK.
checking invariant (9) `Role::MaxMembersBound': OK.
checking invariant (10) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (11) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (12) `Session::NoSessionCycle': OK.
checking invariant (13) `User::MaxSessionsPerUser': OK.
checking invariant (14) `User::NoExclusiveRolesAssigned': FAILED.
checking invariant (14) `User::NoExclusiveRolesAssigned': FAILED.
  -> false : Boolean
checking invariant (15) `User::NoUserCycle': OK.
checked 15 invariants in 0.023s, 1 failure.
1.x> -- Expect: `User::NoExclusiveRolesAssigned': FAILED.
1.x> -- `!set adminEngME.identicalSeniorAllowed := false` :=> causes the overlapping admin and eng roles to become conflicting at senior roles
```

Figure 3: Test Output

Relevant Invariants and Operations:

- `User::NoExclusiveRolesAssigned` enforces the SSD constraint across user assignments.
- `Role::userAssignmentExclusive()` calculates conflict sets, respecting hierarchy and flags.
- `MutuallyExclusive.wrtUserAssignment`, `wrtJuniors`, `wrtSeniors`, and `identicalSeniorAllowed` control SSD applicability and propagation.

2.3 Dynamic Separation of Duties

In Ray et al. [2], dynamic separation of duties (DSD) is enforced through the definition of role pairs that should not be active within the same session. A DSD constraint is checked at runtime by evaluating whether any two conflicting roles are activated concurrently. This prevents a user from exercising conflicting permissions during a single session, even if those roles were both assigned to the user.

The extended model in Kuhlmann et al. [1] redefines DSD enforcement by associating it with the `MutuallyExclusive` class. Each mutual exclusion relationship may optionally activate DSD behavior by setting the `wrtActiveRoles`

flag to `true`. If set, the model checks for activation-time violations at each snapshot, evaluating whether conflicting roles are simultaneously active.

Propagation of the conflict to senior or junior roles is further controlled using the `wrtSeniors` and `wrtJuniors` flags. For example, if role A and role B are mutually exclusive with `wrtActiveRoles = true` and `wrtJuniors = true`, then any juniors of A or B will also be included in the exclusion check.

This allows fine-grained control over dynamic constraints that operate at session-activation level, snapshot granularity, and flexible inheritance direction—contrasting the global, fixed semantics of the earlier model. Just like SSD, DSD is now determined per mutual exclusion and not globally for all roles.

Relevant Invariants and Operations:

- `Session::NoExclusiveActiveRolesPerSnapshot` enforces the DSD check at each snapshot.
- `Role::activeRolesExclusives()` computes conflicting roles, accounting for propagation flags.
- `MutuallyExclusive.wrtActiveRoles`, `wrtJuniors`, and `wrtSeniors` determine whether a given conflict should apply and how it should propagate through the role hierarchy.

3 History-Based and Resource-Based Dynamic Separation of Duties

Ray et al.'s [2] work was extended upon in Kuhlmann et al. [1] to provide two additional constructs: Resource-based dynamic separation of duties and history-based dynamic separation fo duties. As with other additions made by Kuhlmann et al., these additional constructs allow finer-grained control over the permission landscape of an application. Specifically, they look at *mutually exclusive* actions and provide constraints ensuring that a single user cannot execute both mutually exclusive actions. The primary difference between these two methods, which we will discuss more about below, is in how they perform their checks.

3.1 Resource-Based Dynamic Separation of Duties

Resource-based dynamic separation of duties (RBDSOD) ensures that no user may perform both mutually exclusive actions within a single session, snapshot, and action. This restricts users from being able to, for instance, create and approve their own documents.

In our tested scenario, we configured a user, `nick`, who receives the relevant role of `eng` (among other roles). Nick then tries to *write* and *approve* his own *schematic* within a single snapshot/session/action grouping. This is rejected due to the RBDSOD constraint. However, if the RBDSOD constraint is removed nick's `checkAccess()` is approved.

As defined in [1] and implemented in our model specification, the RBDSOD constraint is turned on/off through the setting of the `resourceBasedDynamicSeparationOfDuty` flag defined on the `Resource` class. This allows the constraint to be defined per resource and acts in conjuction with mutual exclusion proagation constraints, `wrtJuniors` and `wrtSeniors` which determine constraint propagation direction.

Relevant Invariants and Operations

- `Resource::ResourceBasedDSOD` enforces the RBDSOD constraint based on various settings of flags on the snapshots.
- `Resource::resourceBasedDynamicSepartionOfDuty` provides a flag on the `Resource` which allows for setting or disabling of the RBDSOD constraint.

3.2 History-Based Dynamic Separation of Duty

History-based dynamics separation of duties (HBDSOD) is concerned with mutually exclusive actions across different snapshots. This restricts a user from, for instance, attempting to circumvent RBDSOD by simply starting a new snapshot.

In our tested scenario, `matt`, who receives the relevant role of `eng`, attempts to perform a `write` action on a `schematic`, then in a different snapshot attempts to perform an `approve` action on the `schematic`. As the `schematic` has `historyBasedDynamicSeparationOfDuty` set to `true`, `matt` is unable to perform both actions and the `HistoryBasedDSOD` constraint is flagged FALSE.

As defined in Kulmann et al. [1] and implemented in our model specification, the HBDSOD constraint is turned on/off through the setting of the `historyBasedDynamicSeparationOfDuty` flag defined on the `Resource` class. This allows for the constraint to be defined and applied per resource. The constraint works in conjuction with the `wrtJuniors` and `wrtSeniors` flags to determine constraint propagation direction.

Object Diagram for Test 2 - Resource-Based Dynamic Separation of Duties

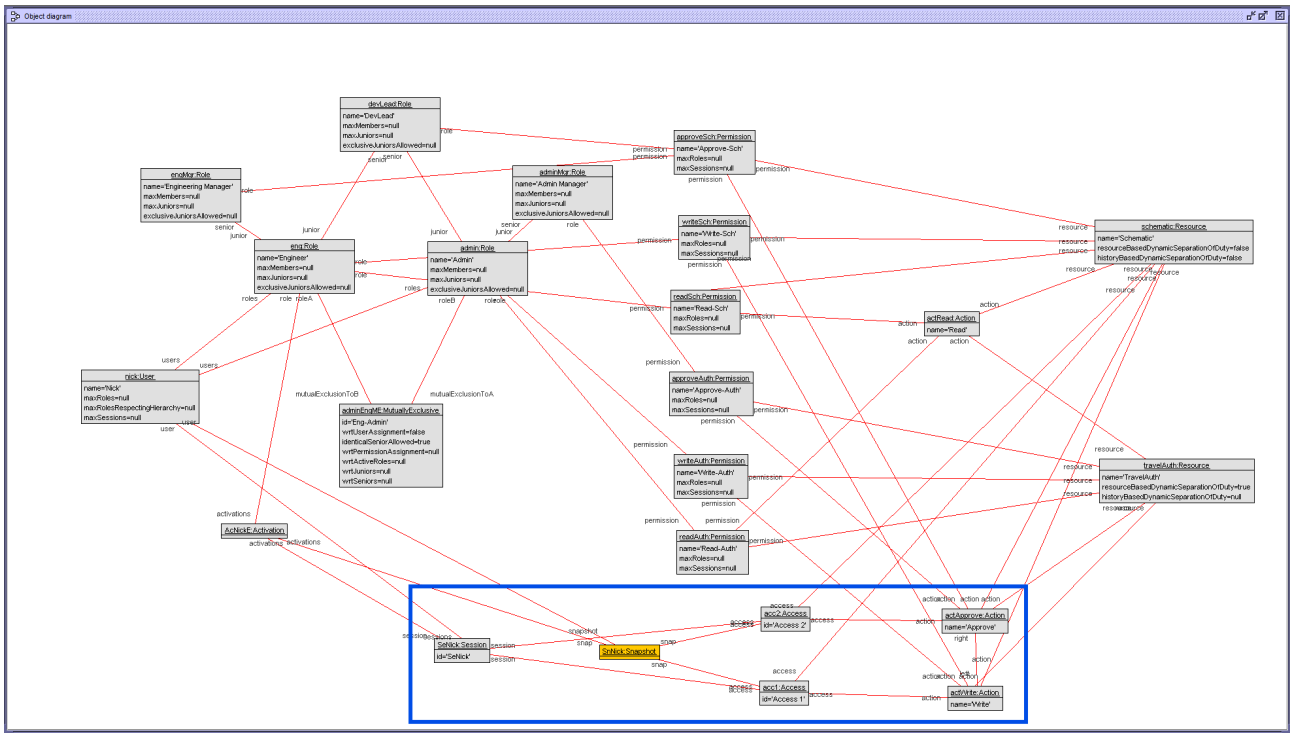

Figure 4: Object Diagram

Test Output for Test 2 - Resource-Based Dynamic Separation of Duties

```
2.x> ------------------------------------------------------------
2.x> -- 9.  RUN THE CONSISTENCY CHECK  --------------------------
2.x> ------------------------------------------------------------
2.x> check
checking structure...
checked structure in 4ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (3) `Permission::PermissionMaxRoles': OK.
checking invariant (4) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (5) `Resource::HistoryBasedDSOD': OK.
checking invariant (6) `Resource::ResourceBasedDSOD': FAILED.
  -> false : Boolean
checking invariant (7) `Role::AcyclicRoleHierarchy': OK.
checking invariant (8) `Role::MaxJuniorsBound': OK.
checking invariant (9) `Role::MaxMembersBound': OK.
checking invariant (10) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (11) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (12) `Session::NoSessionCycle': OK.
checking invariant (13) `User::MaxSessionsPerUser': OK.
checking invariant (14) `User::NoExclusiveRolesAssigned': OK.
checking invariant (15) `User::NoUserCycle': OK.
checked 15 invariants in 0.025s, 1 failure.
2.x> --  Expect: Resource::ResourceBasedDSOD FAILED
2.x>
2.x> ? SeNick.checkAccess(actWrite, travelAuth, SnNick) -- Expect False --> admin is not active in Nick's session
-> false : Boolean
2.x> ? SeNick.checkAccess(actWrite, schematic, SnNick) -- Expect False --> ResourceBasedDSOD blocks
-> false : Boolean
2.x> ? SeNick.checkAccess(actApprove, schematic, SnNick) -- Expect False --> engMgr required
-> false : Boolean
2.x>
2.x> !set schematic.resourceBasedDynamicSeparationOfDuty := false
2.x>
2.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (3) `Permission::PermissionMaxRoles': OK.
checking invariant (4) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (5) `Resource::HistoryBasedDSOD': OK.
checking invariant (6) `Resource::ResourceBasedDSOD': OK.
checking invariant (7) `Role::AcyclicRoleHierarchy': OK.
checking invariant (8) `Role::MaxJuniorsBound': OK.
checking invariant (9) `Role::MaxMembersBound': OK.
checking invariant (10) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (11) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (12) `Session::NoSessionCycle': OK.
checking invariant (13) `User::MaxSessionsPerUser': OK.
checking invariant (14) `User::NoExclusiveRolesAssigned': OK.
checking invariant (15) `User::NoUserCycle': OK.
checked 15 invariants in 0.022s, 0 failures.
2.x>
2.x> ? SeNick.checkAccess(actWrite, schematic, SnNick) -- Expect True --> Reversed RBDSOD
-> true : Boolean
2.x>
2.x> -- Expect: 0 failures.
2.x> -- `!set schematic.resourceBasedDynamicSeparationOfDuty := false` disables RBDSOD on the schematic.
```

Figure 5: Test Output

Relevant Invariants and Operations:

- `User::NoExclusiveRolesAssigned` enforces the SSD constraint across user assignments.
- `Role::userAssignmentExclusive()` calculates conflict sets, respecting hierarchy and flags.
- `MutuallyExclusive.wrtUserAssignment`, `wrtJuniors`, `wrtSeniors`, and `identicalSeniorAllowed` control SSD applicability and propagation.

Object Diagram for Test 3 - History-Based Dynamic Separation of Duties

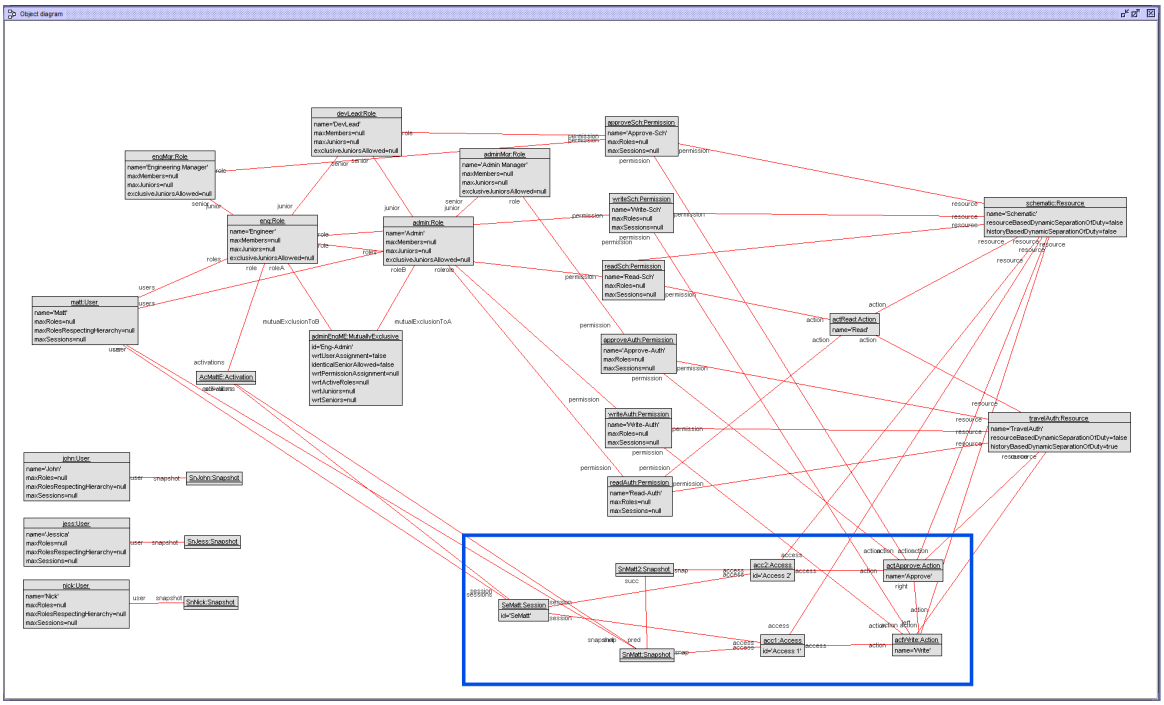

Figure 6: HBDSOD Object Diagram.

Test Output for Test 3 - History-Based Dynamic Separation of Duties

```
3.x> ------------------------------------------------------------
3.x> -- 10.  RUN THE CONSISTENCY CHECK  --------------------------
3.x> ------------------------------------------------------------
3.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (3) `Permission::PermissionMaxRoles': OK.
checking invariant (4) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (5) `Resource::HistoryBasedDSOD': FAILED.
  -> false : Boolean
checking invariant (6) `Resource::ResourceBasedDSOD': OK.
checking invariant (7) `Role::AcyclicRoleHierarchy': OK.
checking invariant (8) `Role::MaxJuniorsBound': OK.
checking invariant (9) `Role::MaxMembersBound': OK.
checking invariant (10) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (11) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (12) `Session::NoSessionCycle': OK.
checking invariant (13) `User::MaxSessionsPerUser': OK.
checking invariant (14) `User::NoExclusiveRolesAssigned': OK.
checking invariant (15) `User::NoUserCycle': OK.
checked 15 invariants in 0.028s, 1 failure.
3.x> --  Expect: Resource::HistoryBasedDSOD FAILED
3.x>
3.x> ? SeMatt.checkAccess(actWrite, travelAuth, SnMatt) -- Expect False --> admin is not active in Matt's session
-> false : Boolean
3.x> ? SeMatt.checkAccess(actWrite, schematic, SnMatt) -- Expect False --> HBDSOD blocks
-> false : Boolean
3.x> ? SeMatt.checkAccess(actApprove, schematic, SnMatt) -- Expect False --> engMgr required
-> false : Boolean
3.x>
3.x> !set schematic.historyBasedDynamicSeparationOfDuty := false
3.x>
3.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (3) `Permission::PermissionMaxRoles': OK.
checking invariant (4) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (5) `Resource::HistoryBasedDSOD': OK.
checking invariant (6) `Resource::ResourceBasedDSOD': OK.
checking invariant (7) `Role::AcyclicRoleHierarchy': OK.
checking invariant (8) `Role::MaxJuniorsBound': OK.
checking invariant (9) `Role::MaxMembersBound': OK.
checking invariant (10) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (11) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (12) `Session::NoSessionCycle': OK.
checking invariant (13) `User::MaxSessionsPerUser': OK.
checking invariant (14) `User::NoExclusiveRolesAssigned': OK.
checking invariant (15) `User::NoUserCycle': OK.
checked 15 invariants in 0.032s, 0 failures.
3.x>
3.x> ? SeMatt.checkAccess(actWrite, schematic, SnMatt) --> Expect True
-> true : Boolean
3.x>
3.x> -- Expect: 0 failures.
3.x> -- `!set schematic.historyBasedDynamicSeparationOfDuty := false` disables HBDSOD on the schematic.
```

Figure 7: Test Output

Relevant Invariants and Operations:

- `User::NoExclusiveRolesAssigned` enforces the SSD constraint across user assignments.

- `Role::userAssignmentExclusive()` calculates conflict sets, respecting hierarchy and flags.
- `MutuallyExclusive.wrtUserAssignment`, `wrtJuniors`, `wrtSeniors`, and `identicalSeniorAllowed` control SSD applicability and propagation.

4 Validation Discussion

Validation around the functionality of the history-based, resource-based, and role hierarchies relationships were focused on during the validation phase of the project. While validation went relatively smoothly, a few notable issues arose during this phase:

1. `identicalSeniorsAllowed`: During validation it was discovered that upon setting the `identicalSeniorsAllowed` flag, all SSD/DSD relationships were ignored in constraints. This was due to the traversal method originally used which only considered the roles directly assigned to the user. However, this is not the desired behavior. To resolve this issue we needed to ensure that the starting point of the traversal for the constraint started with the transitive closure of the role and all of its junior and senior roles.
2. `User::AssignRole`: Original `noSSDChild` constraint checks did not properly traverse all relevant classes and properties. As a result, the assign role did not properly meet requirements. To resolve the condition, additional relevant considerations such as junior transitivity were conjoined using the `and` operator.

   After final testing, there were no remaining issues discovered within the scenarios that we developed. All validations performed as expected with expected outputs. Role hierarchies, resource-based dynamic separation of duties, and history-based dynamic separation of duties was focused upon heavily in validation and scenario cases to validate and demonstrate the functionality discussed in Kuhlmann et al. [1].

While there were many tool comfortability issues along the way that are attributed to the newness of the tool to this team, most issues were resolved via trial and error or luck. For example, during initial development the question of SOIL vs query method bodies arose. SOIL bodies would have provided us with a more detailed sequence diagram output. However, it also introduces significant complexity into the models construction. Due to no instruction requiring a sequence diagram output, we opted to define method bodies using the query method approach.

Additional issues with the tool were discovered along the way. For instance, how USE treats SOIL method calls versus query method calls. While many of these issues did not necessarily make it into the final product, as design direction changed, they assisted greatly in our understanding of the tool's functionality and desired syntax.

All figures and screenshots are located in the Appendix or displayed and discussed in the paper's body.

5 Issues Encountered During Development

During the initial phase of development, we discovered `!openter` could not be used on operations with declared bodies. Thus, in order to still use `!openter`, we created Helper operations containing the desired body functionality while keeping the properly named operations void of such details. This allowed use of `!openter` during constraint testing but was not an optimal solution. This structure resulted in named operations added to the model solely for the purpose of navigating the use tool and that had no business purpose for existing. During later phases of development, we transitioned to the structure as currently submitted and discussed in this paper, defining operations as query operations.

6 Summary of Contributions

Work for this project was divided evenly with each team member contributing equally to the final deliverable product. While this project was primarily a single file, team members worked by splitting the code into two separate copies of the final product so each team member could would independently without conflicts in git repositories.

Due to the single-file nature of the USE tool, with exception for scenario development, collaboration was performed by making individual copies of the working model and adding collective work to the sole final model file. In this way we were able to minimize conflicts during our work.

While work was heavily equally split, with each team member providing full write-ups of the models individually and then combining works into a final product, there are some tasks where individual focus was split between members.

- Matt: While work was heavily performed equally, Matt focused on invariant scenario generation, ensuring that invariants were covered and minor issues were resolved.
- Nick: While work was heavily performed equally, Nick focused on project documentation, ensuring consistent screenshots and figures for this report.

During the project's development, there were a number of team meeting to ensure alignment with the intent of the design and to incorporate concepts and works from each others developments. Additionally, asynchronous work was performed within a private GitHub repository.

Estimated Effort:

- **Matt: 50%**
- **Nick: 50%**

7 Location-Aware RBAC Extensions

Following the completion of our RBAC model based on Kuhlmann et al.[1], we extended the system to incorporate location-aware access control as specified by Ray et al.[5]. This extension transforms our RBAC model into a Location-aware Role-Based Access Control (LRBAC) model, enabling fine-grained access control based on the physical or logical location of users, roles, and resources. The location-aware extensions required modifications to the core RBAC model while maintaining backward compatibility with existing role hierarchies and separation-of-duty constraints. Our approach integrates spatial relationships and location hierarchies directly into the access control decision-making process.

New Artifacts Introduced for Location Support:

- **Location**: Represents physical or logical locations with hierarchical containment relationships
- **UserAt**: Tracks user locations at specific snapshots in time
- **Location-based constraints**: Role assignment, activation, and permission location restrictions

Updated - LRBAC Class Diagram

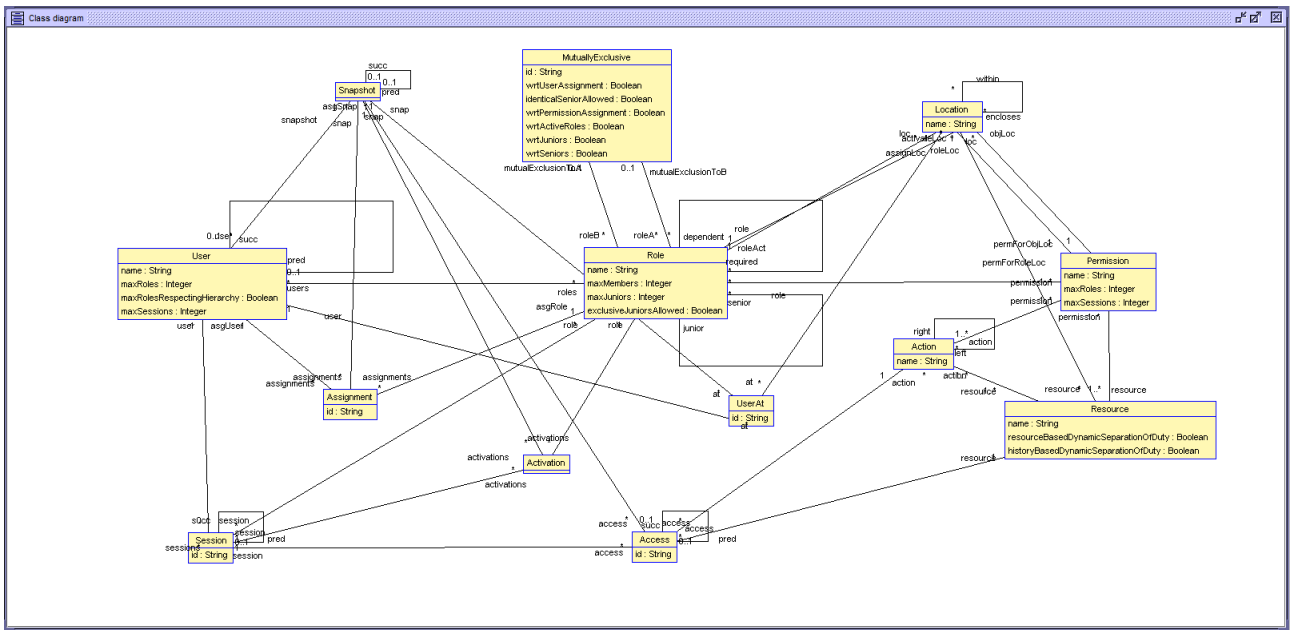

Figure 8: LRBAC - Class Diagram

7.1 Impact of Location on RBAC Components

7.1.1 Location Hierarchy and Spatial Relationships

Our model implements a comprehensive location hierarchy using the `Location` class with `LocationHierarchy` associations. The hierarchy supports containment relationships where locations can enclose other locations, forming a tree-like structure with a "universe" location at the root. The location hierarchy enables inheritance of location-based permissions, where access granted at a higher-level location automatically applies to all contained sub-locations. This is implemented through the `ancestors()` and `descendants()` operations that use transitive closure to navigate the hierarchy.

Key Location Operations:

- `Location::ancestors()`: Returns all locations that contain the current location
- `Location::descendants()`: Returns all locations contained within the current location

7.1.2 Users and Location Tracking

Users are tracked through the `UserAt` association, which records a user's location at each snapshot. This temporal location tracking enables location-based authentication where users must be in authorized locations to perform certain actions, dynamic location updates where user locations can change between snapshots, and location verification where the system validates user locations before granting access. The constraint `ExactlyOneLocationPerUser` ensures that each user has exactly one location per snapshot, maintaining location consistency across the system.

7.1.3 Roles and Location Constraints

Roles are enhanced with two distinct location-based constraints: **Assignment Location** (`assignLoc` association) which specifies where users can be assigned to a role, and **Activation Location** (`activateLoc` association) which specifies where users can activate a role in sessions. These constraints are enforced through `AssignedRolesRespectAssignLocation` ensuring users can only be assigned roles if they are in appropriate locations, and `ActivationLocationAllowed` ensuring role activations in a session must occur from authorized locations.

7.1.4 Permissions and Spatial Access Control

Permissions incorporate dual location constraints: **Role Location** (`roleLoc` association) specifying allowable locations for the role exercising the permission, and **Object Location** (`objLoc` association) specifying allowable locations for the target objects. This dual-constraint system enables fine-grained spatial access control where both the user's location and the resource's location must satisfy permission requirements.

7.1.5 Objects and Location Association

All objects are associated with specific locations through the `ResourceLocation` association. This enables spatial resource protection where resources can only be accessed from authorized locations, location-based resource discovery where users can find resources based on spatial proximity, and geographic access patterns where different access rules can apply based on resource location.

7.2 Location-Based Access Control Enforcement

7.2.1 Enhanced CheckAccess Operation

The `Session::checkAccess()` operation is significantly enhanced to incorporate location-based constraints. The operation now verifies traditional RBAC permissions through role-based authorization as before, user location constraints ensuring users must be in locations authorized for the role, object location constraints ensuring target objects must be in authorized locations, and spatial containment where location hierarchies are respected through ancestor checking. The access control logic uses intersection of location sets to determine authorization:

```
-- User location must be contained in permission's allowed role-locations
uLocs->exists(lu |
p.roleLoc->includes(lu)
or lu.ancestors()->exists(aLoc | p.roleLoc->includes(aLoc))
)
```

```
and
-- Resource location must be contained in permission's allowed object-locations
rLocs->exists(rl |
p.objLoc->includes(rl)
or rl.ancestors()->exists(oLoc | p.objLoc->includes(oLoc))
)
```

7.2.2 Dynamic Location Updates

The model supports dynamic location changes through the `Assignment` class, which tracks role assignments at specific snapshots and locations. This enables temporal location tracking with assignment history in location context, location-based assignment validation ensuring assignments occur from authorized locations, and audit trails providing complete history of location-based access patterns.

8 Impact of Location on Role Hierarchies and Constraints

8.1 Role Hierarchy with Spatial Context

Location awareness adds spatial context to role hierarchies without fundamentally altering the inheritance structure. Senior roles continue to inherit permissions from junior roles, but location constraints provide additional granularity through spatial inheritance where location constraints can be inherited up and down their own self-contained inheritance structure. Thus, roles individually are associated with the proper location hierarchy level, guaranteeing authorization for locations contained lower in this hierarchy. This allows for a powerful location-aware model without the added complexity of multiple inheritance trees associating themselves to various levels within each other.

8.2 Location-Aware Access Control Test Cases

8.2.1 Test Case 1: Hierarchical Location Access Grant

This test case validates the location hierarchy functionality by demonstrating successful access when both user and object locations satisfy permission requirements through ancestor relationships. The user is located at `Leaf`, the resource is at `Leaf`, and the permission specifies `Root` for both role and object locations. Since `Root` encloses `Leaf` in the location hierarchy, both the user's location and the resource's location satisfy the permission constraints via ancestral containment, resulting in access being granted.

Object Diagram for Test Case 1

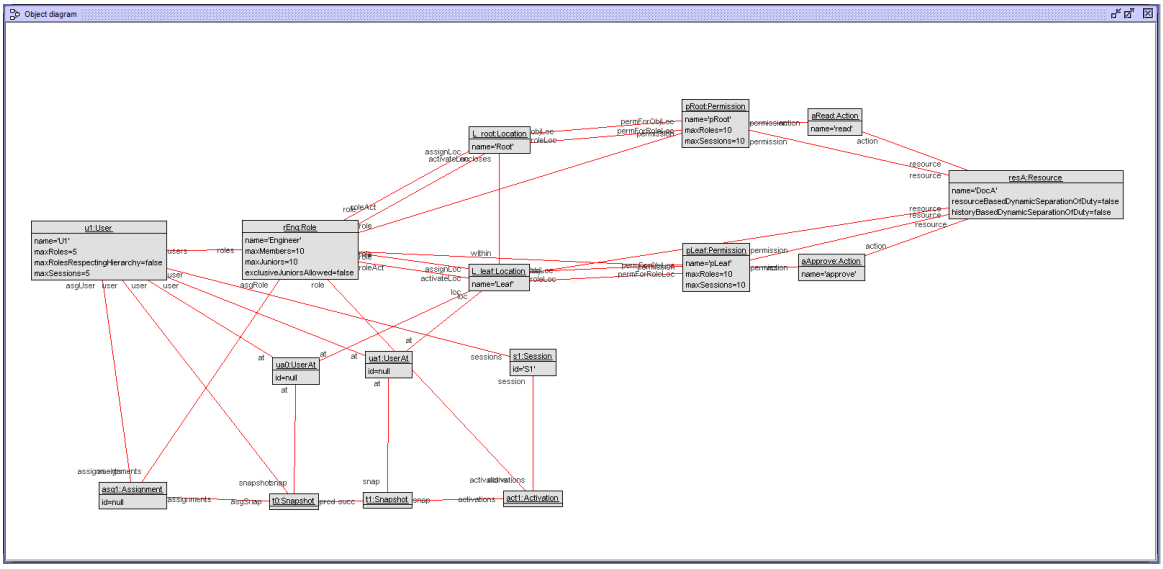

Figure 9: Object Diagram.

Test Output for Test Case 1

```
1.x> check
checking structure...
checked structure in 3ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.037s, 0 failures.
1.x> -- Expect: structure OK; all invariants OK
1.x>
1.x> -- GRANT #1: ancestor-based (user at Leaf, object at Leaf, perm at Root)
1.x> ? s1.checkAccess(aRead, resA, t1)
-> true : Boolean
1.x> -- -> true
1.x>
1.x> -- GRANT #2: exact-location (Leaf for both)
1.x> ? s1.checkAccess(aApprove, resA, t1)
-> true : Boolean
1.x> -- -> true
1.x>
```

Figure 10: Test Output

8.2.2 Test Case 2: User Location Mismatch Denial

This test case demonstrates access denial when the user's location does not satisfy the permission's role location constraints. The user is positioned at `Root`, while the relevant permission requires the role to be exercised from `Leaf` location. Although the resource is correctly located at `Leaf` and matches the permission's object location requirement, the user's location at `Root` fails to satisfy the role location constraint, as `Root` is not contained within or equal to `Leaf`, resulting in access denial.

Object Diagram for Test 2

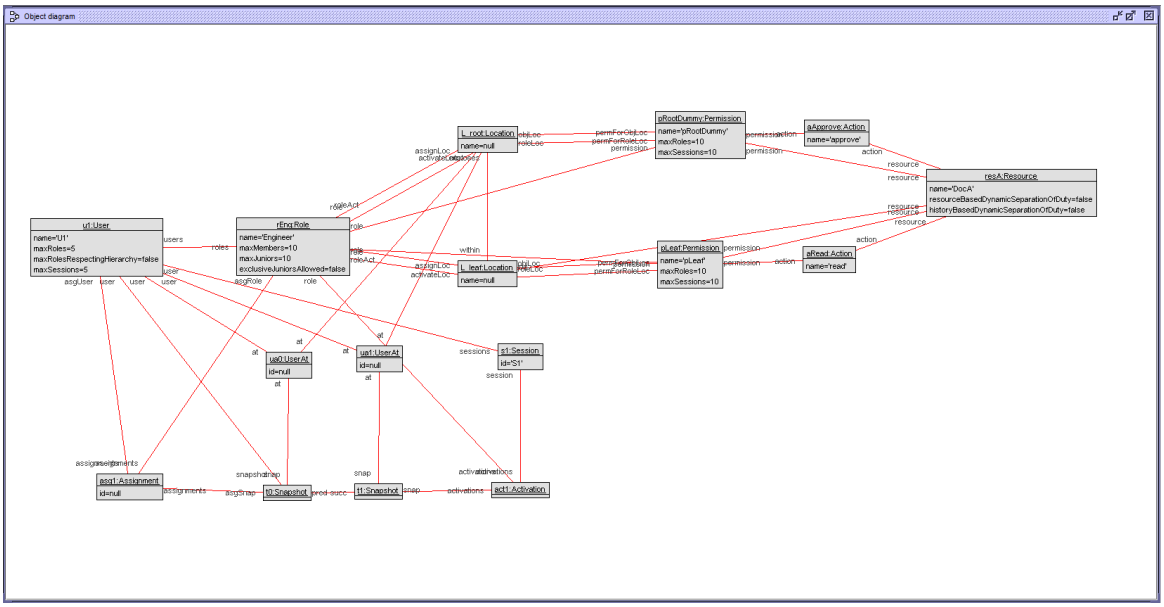

Figure 11: Object Diagram.

Test Output for Test 2

```
2.x> check
checking structure...
checked structure in 2ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.034s, 0 failures.
2.x> -- Expect: all invariants OK
2.x>
2.x> -- DENY: user at Root, perm requires Leaf (no ancestor of Root equals Leaf)
2.x> ? s1.checkAccess(aRead, resA, t1)
-> false : Boolean
2.x> -- -> false
2.x>
```

Figure 12: Test Output

8.2.3 Test Case 3: Object Location Mismatch Denial

This test case validates access denial when the resource's location does not satisfy the permission's object location constraints. The user is correctly positioned at `Leaf`, and the permission requires both role and object locations to be `Leaf`. However, the target resource is located at `Root`, which does not satisfy the permission's object location requirement. Since `Root` is not contained within `Leaf`, the object location constraint fails, resulting in access denial despite the user being in the correct location.

Object Diagram for Test 3

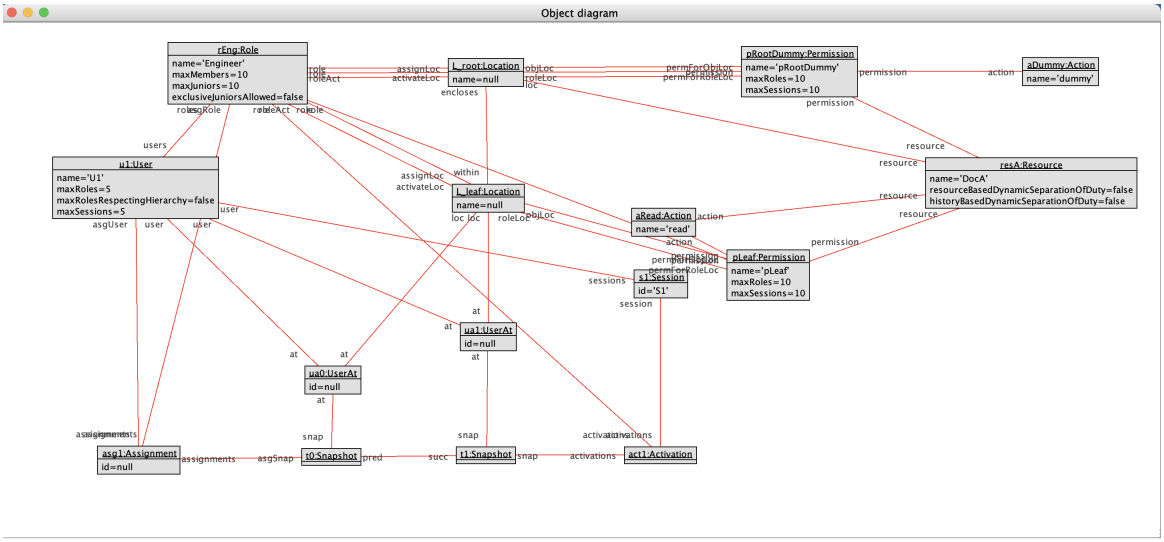

Figure 13: Object Diagram.

Test Output for Test 3

```
3.x> check
checking structure...
checked structure in 0ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.002s, 0 failures.
3.x> -- Expect: all invariants OK
3.x>
3.x> -- DENY: object at Root but pLeaf.objLoc=Leaf (neither Root nor its ancestors are Leaf)
3.x> ? s1.checkAccess(aRead, resA, t1)
-> false : Boolean
3.x> -- -> false
```

Figure 14: Test Output

8.2.4 Test Case 4: Missing Role Activation Denial

This test case demonstrates access denial when a user has been assigned a role but has not activated it in the current session at the queried snapshot. The user possesses the necessary role assignment and is positioned at

the correct location (`Leaf`) for both the permission requirements and resource location. However, no activation exists for the role at snapshot `t1`, meaning the role is not active in the session at the time of the access request. This results in access denial as the `checkAccess` operation requires active roles, not merely assigned roles.

Object Diagram for Test 4

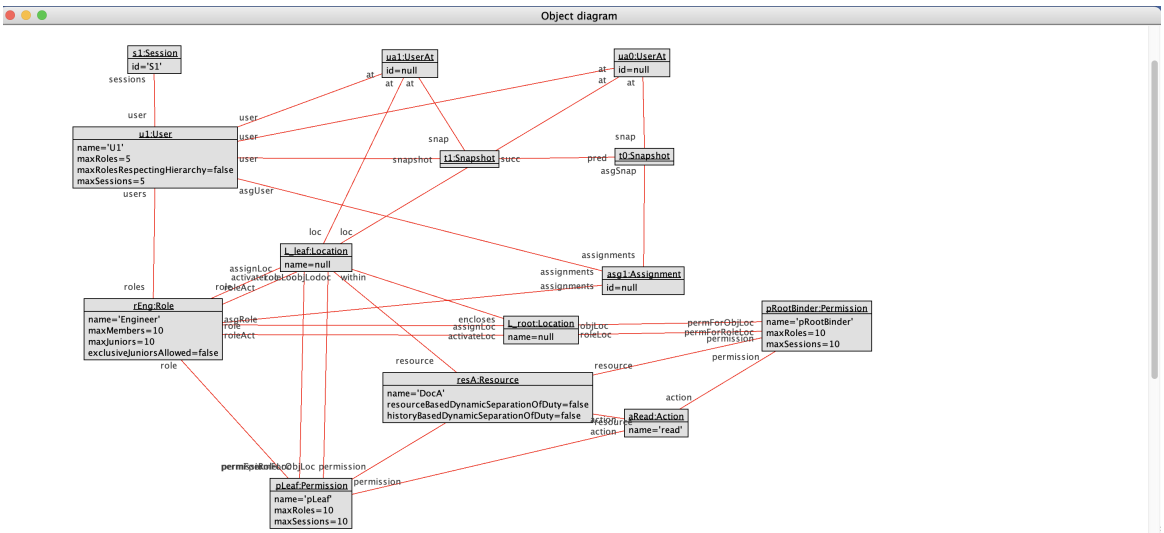

Figure 15: Object Diagram.

Test Output for Test 4

```
4.x> check
checking structure...
checked structure in 0ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.001s, 0 failures.
4.x> -- Expect: all invariants OK
4.x>
4.x> -- DENY: no active role at t1
4.x> ? s1.checkAccess(aRead, resA, t1)
-> false : Boolean
4.x> -- -> false
```

Figure 16: Test Output

8.2.5 Test Case 5: Resource-Based Dynamic Separation of Duties

This test case validates Resource-Based Dynamic Separation of Duties (RBDSOD) enforcement within a single snapshot. The test involves mutually exclusive actions (`approve` and `review`) defined through the

`MutuallyExclusiveActions` association. Initially, access to `approve` is granted as no conflicting actions have been recorded. However, after recording an access to the mutually exclusive `review` action at the same snapshot, subsequent access attempts to `approve` are denied due to the RBDSOD constraint. Non-conflicting actions like `read` remain accessible throughout the test.

Object Diagram for Test 5

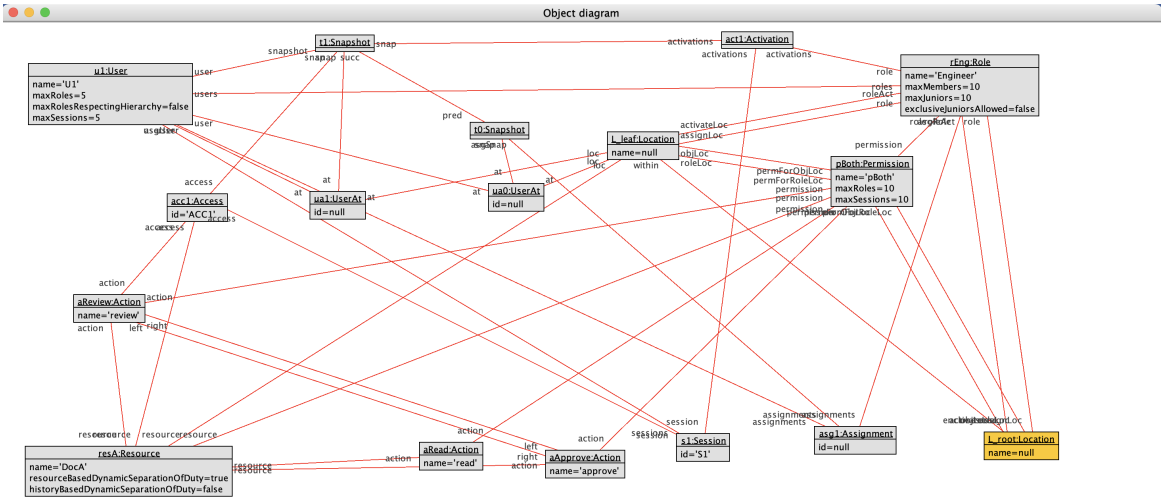

Figure 17: Object Diagram.

Test Output for Test 5

```
5.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.002s, 0 failures.
5.x> -- Expect: all invariants OK
5.x>
5.x> -- GRANT (no conflict recorded yet)
5.x> ? s1.checkAccess(aApprove, resA, t1)
-> true : Boolean
5.x> -- -> true
5.x>
5.x> -- Record a conflicting action at the same snapshot
5.x> !create acc1 : Access
5.x> !set acc1.id := 'ACC1'
5.x> !insert (s1, acc1)     into ActiveAccess
5.x> !insert (resA, acc1)   into AccessResource
5.x> !insert (aReview, acc1) into ActionAccess
5.x> !insert (t1, acc1)     into AccessSnapshot
5.x>
5.x> -- DENY now (RB-DSOD blocks the mutually-exclusive action)
5.x> ? s1.checkAccess(aApprove, resA, t1)
-> false : Boolean
5.x> -- -> false
5.x>
5.x> -- Non-conflicting action still allowed
5.x> ? s1.checkAccess(aRead, resA, t1)
-> true : Boolean
5.x> -- -> true
```

Figure 18: Test Output

8.2.6 Test Case 6: History-Based Dynamic Separation of Duties

This test case demonstrates History-Based Dynamic Separation of Duties (HBDSOD) enforcement across multiple snapshots. A user performs a `review` action on a resource at snapshot `t1`. When the user later attempts to perform the mutually exclusive `approve` action at snapshot `t2`, access is denied because HBDSOD examines the complete history of actions across the snapshot succession chain. The constraint prevents users from circumventing separation of duties by performing conflicting actions across different time periods, maintaining security across temporal boundaries.

Object Diagram for Test 6

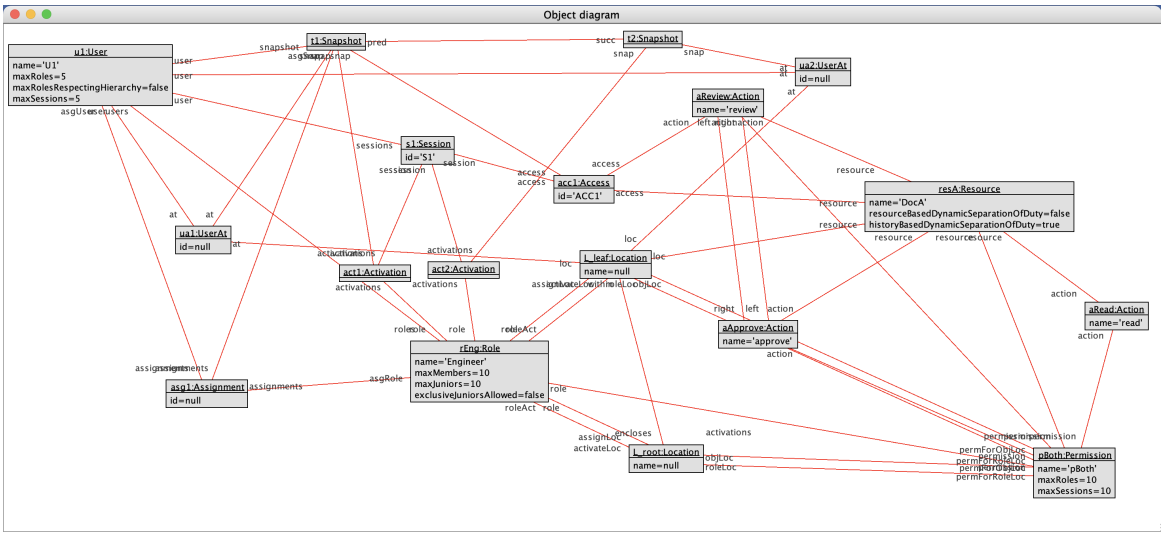

Figure 19: Object Diagram.

Test Output for Test 6

```
6.x> check
checking structure...
checked structure in 1ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.003s, 0 failures.
6.x> -- Expect: structure OK; all invariants OK
6.x>
6.x> -- APPROVE: No conflicting access yet
6.x> ? s1.checkAccess(aApprove, resA, t1)
-> true : Boolean
6.x>
6.x> -- APPROVE: Non-conflicting action
6.x> ? s1.checkAccess(aRead, resA, t1)
-> true : Boolean
6.x>
6.x> -- Record a prior conflicting action at t1
6.x> !create acc1 : Access
6.x> !set acc1.id := 'ACC1'
6.x> !insert (s1,     acc1) into ActiveAccess
6.x> !insert (resA,   acc1) into AccessResource
6.x> !insert (aReview,acc1) into ActionAccess
6.x> !insert (t1,     acc1) into AccessSnapshot
6.x>
6.x> -- DENY at t2 (HB-DSOD sees t1 as predecessor)
6.x> ? s1.checkAccess(aApprove, resA, t2)
-> false : Boolean
6.x> -- -> false
6.x>
6.x> -- Non-conflicting still allowed at t2
6.x> ? s1.checkAccess(aRead, resA, t2)
-> true : Boolean
6.x> -- -> true
```

Figure 20: Test Output

8.2.7 Test Case 7: Ancestral Location Satisfaction Grant

This test case validates the comprehensive ancestor-based location checking mechanism where permissions defined at higher hierarchy levels can be satisfied by entities at lower levels. The scenario establishes a `Root` location that encloses a `Leaf` location, with both the user and resource positioned at `Leaf`. The critical permission specifies `Root` for both role location and object location constraints. The test demonstrates that access is granted because both the user's location (`Leaf`) and the resource's location (`Leaf`) satisfy the permission's requirements through ancestral containment, as `Root` is an ancestor of `Leaf` in the location hierarchy. This validates the transitive closure functionality of the `ancestors()` operation in location-based access control decisions.

Object Diagram for Test 7

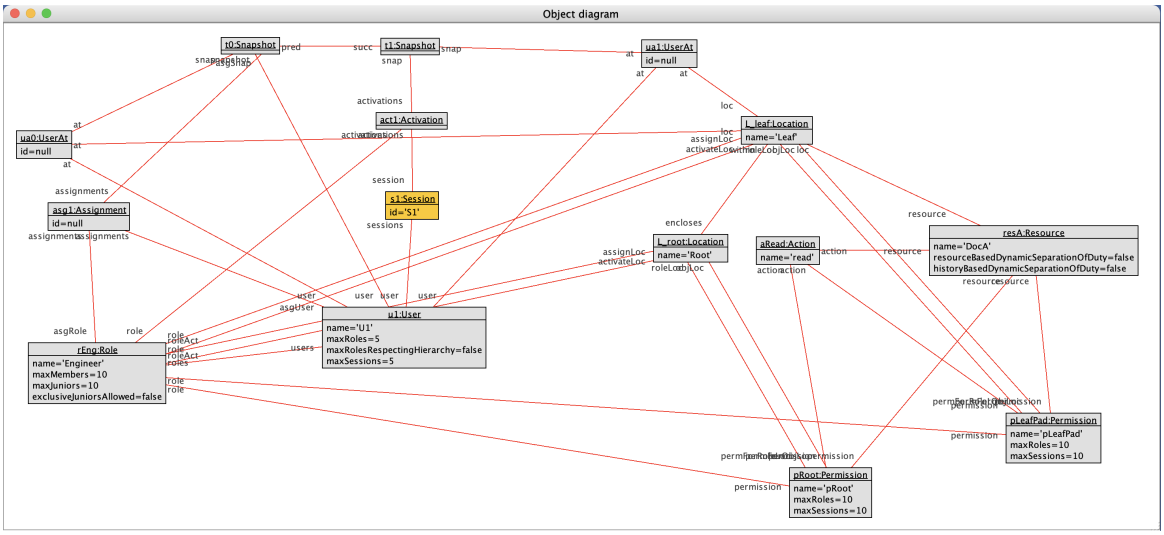

Figure 21: Object Diagram.

Test Output for Test 7

```
7.x> check
checking structure...
checked structure in 0ms.
checking invariants...
checking invariant (1) `Access::NoAccessCycle': OK.
checking invariant (2) `Activation::ActivationLocationAllowed': OK.
checking invariant (3) `Assignment::AssignmentLocationAllowed': OK.
checking invariant (4) `Assignment::AssignmentReflectsMembership': OK.
checking invariant (5) `Location::AcyclicLocationHierarchy': OK.
checking invariant (6) `Permission::NoPermissionToExclusiveRolesTogether': OK.
checking invariant (7) `Permission::PermissionMaxRoles': OK.
checking invariant (8) `Permission::PermissionMaxSessionsPerSnapshot': OK.
checking invariant (9) `Resource::HistoryBasedDSOD': OK.
checking invariant (10) `Resource::ResourceBasedDSOD': OK.
checking invariant (11) `Role::AcyclicRoleHierarchy': OK.
checking invariant (12) `Role::MaxJuniorsBound': OK.
checking invariant (13) `Role::MaxMembersBound': OK.
checking invariant (14) `Role::PrerequisitesRespectedOnAssignment': OK.
checking invariant (15) `Session::NoExclusiveActiveRolesPerSnapshot': OK.
checking invariant (16) `Session::NoSessionCycle': OK.
checking invariant (17) `Snapshot::ExactlyOneLocationPerUser': OK.
checking invariant (18) `User::AssignedRolesRespectAssignLocation': OK.
checking invariant (19) `User::MaxSessionsPerUser': OK.
checking invariant (20) `User::NoExclusiveRolesAssigned': OK.
checking invariant (21) `User::NoUserCycle': OK.
checked 21 invariants in 0.002s, 0 failures.
7.x> -- Expect: structure OK; all invariants OK
7.x>
7.x> -- GRANT: both userLoc (Leaf) and objLoc (Leaf) satisfy pRoot via ancestors to Root
7.x> ? s1.checkAccess(aRead, resA, t1)
-> true : Boolean
7.x> -- -> true
```

Figure 22: Test Output

`CreateSession` and RBML Template Instantiation

RBML template instantiation was performed from Figure 2 in Kuhlmann et al. [1]. All unconstrained multiplicities $|a \to *$, constrained $|o \to 1..*$ and $|q \to 1..*$, with all parameters bound to their given names from Figure 2 in Kuhlmann et al. [1]. It does not activate any roles at creation time. Role activation is performed at a later time through `Session::AddActiveRole`. We validated this functionality by verifying no new roles were created on execution of the `CreateSession` function, showing in Figure 23

```
use> !openter u1 CreateSession()
use> !create ts : Session
use> !insert (ts, u1) into ActiveUser
use> !opexit ts
postcondition `post12' is true
use> ? ts.role->isEmpty()
-> true : Boolean
```

Figure 23: `CreateSession` behavior validation.

We adopted `CreateSession` from [5] and [1]. It returns a newly created `Session` associated with the user through `result.oclIsNew()` and `self.sessions = self.sessions@pre->including(result)`.

Division of Labor - Part II

Team collaboration was conducted in an iterative, primarily asynchronous manner, supplemented by frequent live meets for joint coding sessions. While these meetings were planned weekly, they often occurred more frequently on an ad hoc basis. During asynchronous work, each team member created a copy of a known working state to minimize conflicts in the shared specification file. During live sessions, these individual code segments were merged into a single working specification, which was then copied to continue the next iteration. This process generated numerous intermediate artifacts that do not appear in the final specification, an example of which is shown in Figure 24.

Contribution Percentages:

- Matt: 50%
- Nick: 50%

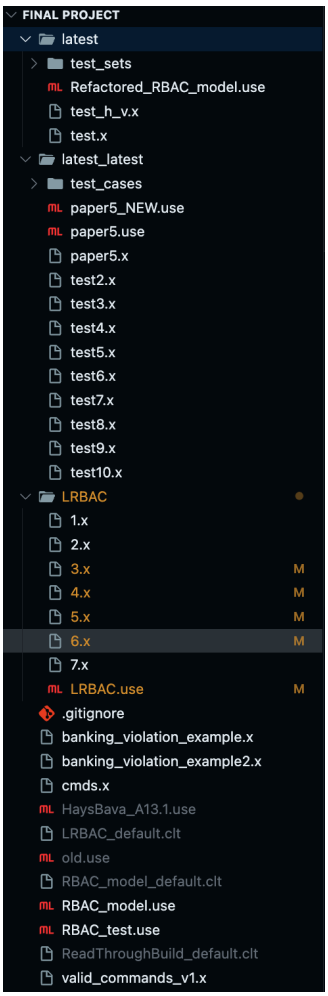

Figure 24: Intermediate artifacts list.

Use of Generative AI

The authors used ChatGPT [6] and Overleaf Writefull [7] only for minor formatting, language refinement, and assistance in interpreting complex sections of referenced papers and mathematical notations; all substantive content is the work of the authors.

Appendix

The original paper drafted for the creation of the model defined by Ray et al. [2] is maintained for completeness and is located within the provided `Bava_Hays_A11.zip` within the root of the .zip.

The following contains verification and validation results from testing and validation performed during the project. As method bodies were defined in many cases as query operations, the `!openter` method is not available. As a result we invoke the query methods with the `?` operator and provide expected outputs in the comments.

Invariant/Constraint Validations and Outputs

```
4.x> !create r1 : Role
4.x> !create r2 : Role
4.x> !create r3 : Role
4.x> !insert (r1, r2) into RoleHierarchy
4.x> !insert (r2, r3) into RoleHierarchy
4.x>
4.x> ? r1.juniors()
-> Set{r2,r3} : Set(Role)
4.x> --> Expect Set{r2,r3}
4.x>
4.x> ? r3.seniors()
-> Set{r1,r2} : Set(Role)
4.x> --> Expect Set{r1,r2}
```

Figure 25: `Role::juniors()` and `Role::seniors()` postcondition validation results.

```
5.x> !create req : Role
5.x> !create dep : Role
5.x> !insert (req, dep) into PrerequisiteRoles
5.x>
5.x> ? dep.required()
-> Set{req} : Set(Role)
5.x> --> Expect Set{req}
```

Figure 26: `Role::required()` postcondition validation results.

```
6.x> !create u1 : User
6.x> !create u2 : User
6.x> !create u3 : User
6.x>
6.x> !insert (u1,u2) into PredSuccUser
6.x> !insert (u2,u3) into PredSuccUser
6.x>
6.x> ? u1.successors()
-> Set{null,u2,u3} : Set(User)
6.x> -- Expect :=> Set{null, u2,u3}
```

Figure 27: `User::successors()` postcondition validation results.

```
7.x> !create s1 : Session
7.x> !create s2 : Session
7.x> !create s3 : Session
7.x> !insert (s1, s2) into PredSuccSession
7.x> !insert (s2, s3) into PredSuccSession
7.x>
7.x> ? s1.successors()
-> Set{null,s2,s3} : Set(Session)
7.x> -- Expect :=> Set{null, s2,s3}
```

Figure 28: `Session::successors()` postcondition validation results.

```
8.x> !create n1 : Snapshot
8.x> !create n2 : Snapshot
8.x> !create n3 : Snapshot
8.x> !insert (n1,n2) into PredSuccSnapshot
8.x> !insert (n2,n3) into PredSuccSnapshot
8.x>
8.x> ? n1.successors()
-> Set{null,n2,n3} : Set(Snapshot)
8.x> -- Expect :=> Set{null,n2,n3}
```

Figure 29: `Snapshot::successors()` postcondition validation results.

```
9.x> -- Valid: simple chain, no cycle
9.x> !create R1 : Role
9.x> !create R2 : Role
9.x> !insert (R1, R2) into RoleHierarchy
9.x> ? not R1.senior->closure(senior)->includes(R1)
-> true : Boolean
9.x> -- Expected: -> true
9.x>
9.x> -- Invalid: introduce cycle R2->R1
9.x> !insert (R2, R1) into RoleHierarchy
9.x> ? not R1.senior->closure(senior)->includes(R1)
-> false : Boolean
9.x> -- Expected: -> false
```

Figure 30: `AcyclicRoleHierarchy` invariant validation results.

```
10.x> !create R : Role
10.x> !set R.maxMembers := 2
10.x> !create U1 : User
10.x> !create U2 : User
10.x>
10.x> -- Valid: exactly at bound
10.x> !insert (U1, R) into UserAssignment
10.x> !insert (U2, R) into UserAssignment
10.x> ? R.users->size() <= R.maxMembers
-> true : Boolean
10.x> -- Expected: -> true
10.x>
10.x> -- Invalid: exceed bound
10.x> !create U3 : User
10.x> !insert (U3, R) into UserAssignment
10.x> ? R.users->size() <= R.maxMembers
-> false : Boolean
10.x> -- Expected: -> false
```

Figure 31: `MaxMembersBound` invariant validation results.

```
11.x> !create R1 : Role
11.x> !create R2 : Role
11.x> !create R3 : Role
11.x> !set    R1.maxJuniors := 1
11.x>
11.x> -- Valid: only one junior in closure
11.x> !insert (R1, R2) into RoleHierarchy
11.x> ? R1.junior->closure(junior)->size() <= R1.maxJuniors
-> true : Boolean
11.x> -- -> true : Boolean
11.x>
11.x> -- Invalid: add a second junior
11.x> !insert (R1, R3) into RoleHierarchy
11.x> ? R1.junior->closure(junior)->size() <= R1.maxJuniors
-> false : Boolean
11.x> -- -> false : Boolean
```

Figure 32: `MaxJuniorsBound` invariant validation results.

```
12.x> !insert (S1, U) into ActiveUser
12.x> ? U.sessions->size() <= U.maxSessions
-> true : Boolean
12.x> -- -> true : Boolean
12.x>
12.x> -- Invalid: two sessions
12.x> !create S2 : Session
12.x> !insert (S2, U) into ActiveUser
12.x> ? U.sessions->size() <= U.maxSessions
-> false : Boolean
12.x> -- -> false : Boolean
```

Figure 33: `MaxSessionsPerUser` invariant validation results.

```
13.x> !create P : Permission
13.x> !set P.maxRoles := 1
13.x> !create R1 : Role
13.x> !create R2 : Role
13.x>
13.x> -- Valid: one role assigned
13.x> !insert (R1, P) into PermissionAssignment
13.x> ? P.role->size() <= P.maxRoles
-> true : Boolean
13.x> -- Expected: -> true
13.x>
13.x> -- Invalid: two roles
13.x> !insert (R2, P) into PermissionAssignment
13.x> ? P.role->size() <= P.maxRoles
-> false : Boolean
13.x> -- Expected: -> false
```

Figure 34: `PermissionMaxRoles` invariant validation results.

```
14.x> -- Valid: one activation (session) at Snap1 for P
14.x> !insert (S1,    A1) into ActivationSession
14.x> !insert (R,     A1) into ActivationRole
14.x> !insert (Snap1, A1) into ActivationSnapshot
14.x> ? Session.allInstances()->select(s | s.activations->exists(a | a.snap = Snap1 and a.role.permission->includes(P)))->size() <= P.maxSessions
-> true : Boolean
14.x> -- -> true : Boolean
14.x>
14.x> -- Invalid: add second activation (session) at Snap1 for P
14.x> !insert (S2,    A2) into ActivationSession
14.x> !insert (R,     A2) into ActivationRole
14.x> !insert (Snap1, A2) into ActivationSnapshot
14.x> ? Session.allInstances()->select(s | s.activations->exists(a | a.snap = Snap1 and a.role.permission->includes(P)))->size() <= P.maxSessions
-> false : Boolean
14.x> -- -> false : Boolean
```

Figure 35: `PermissionMaxSessionsPerSnapshot` invariant validation results.

```
15.x> !create ME : MutuallyExclusive
15.x> !set    ME.wrtUserAssignment       := false
15.x> !set    ME.wrtPermissionAssignment := true
15.x> !create R1 : Role
15.x> !create R2 : Role
15.x> !insert (R1, ME) into MutuallyExclusiveRoleA
15.x> !insert (R2, ME) into MutuallyExclusiveRoleB
15.x> !create P1 : Permission
15.x> !insert (R1, P1) into PermissionAssignment
15.x>
15.x> ? P1.role->forAll(r1, r2 | r1 <> r2 implies not r1.permissionAssignmentExclusive()->includes(r2))
-> true : Boolean
15.x> -- -> true : Boolean
```

Figure 36: `NoPermissionTogetherExclusiveRoleTogether` invariant validation results.

```
16.x> -- Valid: S1->S2
16.x> !create S1 : Session
Error: An object with name `S1' already exists.
16.x> !create S2 : Session
Error: An object with name `S2' already exists.
16.x> !insert (S1, S2) into PredSuccSession
Error: Link `PredSuccSession' between (S1,S2) already exist.
16.x> ? not S1.succ->asSet()->closure(succ)->includes(S1)
-> false : Boolean
16.x> -- Expected: -> true
16.x>
16.x> -- Invalid: S2->S1
16.x> !insert (S2, S1) into PredSuccSession
Error: Link `PredSuccSession' between (S2,S1) already exist.
16.x> ? not S1.succ->asSet()->closure(succ)->includes(S1)
-> false : Boolean
16.x> -- Expected: -> false
```

Figure 37: `NoSessionCycle` invariant validation results.

```
17.x> -- Valid: X1->X2
17.x> !create X1 : Access
17.x> !create X2 : Access
17.x> !insert (X1, X2) into PreSuccAccess
17.x> ? not X1.succ->asSet()->closure(succ)->includes(X1)
-> true : Boolean
17.x> -- Expected: -> true
17.x>
17.x> -- Invalid: X2->X1
17.x> !insert (X2, X1) into PreSuccAccess
17.x> ? not X1.succ->asSet()->closure(succ)->includes(X1)
-> false : Boolean
17.x> -- Expected: -> false
```

Figure 38: `NoAccessCycle` invariant validation results.

References

[1] M. Kuhlmann, K. Sohr, and M. Gogolla, "Employing uml and ocl for designing and analysing role-based access control," *Mathematical Structures in Computer Science*, vol. 23, no. 4, pp. 796–833, 2013.

[2] I. Ray, N. Li, R. France, and D.-K. Kim, "Using uml to visualize role-based access control constraints," in *Proceedings of the ninth ACM symposium on Access control models and technologies*, pp. 115–124, 2004.

[3] K. Sohr, G.-J. Ahn, M. Gogolla, and L. Migge, "Specification and validation of authorisation constraints using uml and ocl," in *European Symposium on Research in Computer Security*, pp. 64–79, Springer, 2005.

[4] W. Alvarez, "Role-based access control: an overview." Presentation, National Institute of Standards and Technology (NIST) Computer Security Resource Center, Aug. 2003.

[5] I. Ray, M. Kumar, and L. Yu, "LRBAC: A Location-Aware Role-Based Access Control Model," in *Proceedings of the International Conference on Information Systems Security (ICISS 2006)* (A. Bagchi and V. Atluri, eds.), vol. 4332 of *Lecture Notes in Computer Science*, (Berlin, Heidelberg), pp. 147–161, Springer-Verlag, 2006.

[6] OpenAI, "Chatgpt," 2025. Large language model used for drafting, reviewing, and refining text and code. Accessed via OpenAI ChatGPT interface.

[7] Writefull, "Writefull for overleaf," 2025. AI-powered writing and language feedback tool integrated with Overleaf for academic and technical writing.